

Chapter 1 (Book) / Chapter 2 (Repo): A Brief Introduction to C++

Source: C++ High Performance — Björn Andrist & Viktor Sehr — Packt — pages 7–26 **Code repo (2nd ed):** [PacktPublishing/Cpp-High-Performance-Second-Edition/Chapter01](https://github.com/PacktPublishing/Cpp-High-Performance-Second-Edition/tree/master/Chapter01) (<https://github.com/PacktPublishing/Cpp-High-Performance-Second-Edition/tree/master/Chapter01>). **Repo numbering note:** The Educative course / this repo inserts a "Getting Started" prelude as Chapter 1. The book's Chapter 1 starts here in `02-brief-introduction-to-cpp/`.

BFS Layer 0 — One-sentence chapter thesis

C++ is the only mainstream language that simultaneously offers **zero-cost abstractions**, **value semantics with strict ownership**, **deterministic destruction (RAII)**, and **const-correctness** — the four properties that let you write code that is *both* fast *and* hard to misuse.

BFS Layer 1 — The seven major sections

#	Section	Pages	One-liner
1	Why C++?	7–10	Three pillars: zero-cost abstractions, portability, robustness.
2	Zero-cost abstractions	7–10	Abstractions in C++ collapse at compile time; in Java/C# they cost runtime.
3	C++ vs other languages (performance)	11–15	Bytecode vs native; heap-only vs stack+heap; GC vs RAII; contiguous vs indirected layout.
4	Non-performance language features	15–20	Value semantics; const correctness; references-as-non-null.
5	Drawbacks of C++	20–21	Compile times, headers, no rich stdlib (yet).
6	Class interfaces & exceptions	21–25	Strict interfaces; copy-and-swap; RAII; exceptions vs error codes.
7	Libraries used in this book	25–26	Boost (header-only) where stdlib is insufficient.

BFS Layer 2 — Concept-by-concept drill-down

1. Zero-cost abstractions

- **Hamlet problem** — same task in C (`struct + raw pointer + strcmp + for`) vs C++ (`std::list<std::string> + std::count`). Both compile to roughly the same machine code, but the C++ version expresses *intent*.
- **Abstraction stack inside C++:**
 - classes ← C-structs + free functions
 - polymorphism ← function pointers
 - lambdas ← classes (compiler-synthesised functor)
 - templates ← code generation
- **The promise:** pay only for what you use; what you don't use costs nothing.

2. Memory layout (C++ vs Java)

- **Scalar objects** — C++ puts them on the **stack** by default; Java boxes everything on the **heap**.
- **Containers** — `std::vector<Car>` stores the **actual objects** contiguously; Java's `ArrayList<Car>` stores **references** to heap-allocated objects.
- **Implication:** 7 elements = 1 allocation in C++, 8 allocations in Java (1 array + 7 objects).
- **Iteration:** contiguous layout → linear cache prefetches → 5–20× faster traversal under cold-cache pressure.
- **Why this matters for HFT:** the hot path of an order book or feed handler iterates contiguous buffers thousands of times per microsecond. Pointer-chasing is a death sentence.

3. Value semantics

- Pass-by-value is the **default**; reference semantics is opt-in (`&`, `*`, smart-ptr).
- **Bagel example** — passing a `std::set<std::string>` by value into `Bagel` *copies* the toppings. Mutating the original set leaves all `Bagels` untouched.
- In Java the same code silently shares the set across all `Bagels` — a class of bugs C++ makes impossible at the type level.
- **Trade-off:** copies are visible in the source — and the compiler will elide them where possible (RVO, NRVO, move).

4. Const correctness

- A `const` member function promises *not to mutate* the object — enforced at compile time.
- A `const T&` parameter promises the *callee* won't mutate — enforced at compile time.
- **Const-overloading pattern:** `auto& leader() const { ... }` returns immutable view; `auto& leader() { ... }` returns mutable view. Same name, picked by the constness of `*this`.

- **Why this matters:** in a low-latency code base, knowing that a function will not touch shared state by *reading the signature alone* eliminates a whole category of data-race & aliasing bugs.

5. References vs pointers in signatures

- $T\& \Rightarrow$ "object is required, cannot be null" — caller can pass by value, callee cannot rebind.
- $T* \Rightarrow$ "object is optional, may be null" — callee must check.
- The signature itself becomes documentation. No annotations, no `@NonNull`, no runtime NPE.

6. Object ownership

- Strict ownership $\approx$ each object has exactly one owner. Express it with:
 - **unique ownership** $\rightarrow$ `std::unique_ptr<T>` or by-value member
 - **shared ownership** $\rightarrow$ `std::shared_ptr<T>` (ref-counted, ~ 2 atomic ops per copy/destroy)
 - **non-owning observation** $\rightarrow$ `raw T*`, `T&`, `std::weak_ptr<T>`
- *Emulating Java's GC by wrapping everything in `shared_ptr` works syntactically but loses cache locality, predictability and adds atomic overhead.*

7. Strict class interfaces

- **Boat/Engine bug:** a class holding `std::shared_ptr<Engine>` is *trivially copyable* by default — copies silently share state, mutations leak.
- **Fix:** explicitly delete copy ops or make the class move-only:

```
Boat(const Boat&)           = delete;
Boat& operator=(const Boat&) = delete;
```

- Rule: copying must be either a **deep copy** or a **compile error** — never a silent alias.

8. Exception safety — the three levels

Guarantee	Meaning	Example
Basic	No leaks; invariants hold; object may be in any valid state.	Naïve <code>op=</code> that copies field-by-field.
Strong	If the operation throws, state is <i>unchanged</i> (commit-or-rollback).	Copy-and-swap <code>op=</code> .
No-throw	Cannot fail. Marked <code>noexcept</code> .	<code>swap</code> , destructors, move ops on well-behaved types.

9. Copy-and-swap idiom

```

auto& operator=(const OakTree& other) {
    auto leafs    = other.leafs_;    // may throw — but *this is untouched
    auto branches = other.branches_; // may throw — but *this is untouched
    std::swap(leafs_,    leafs_);    // no-throw
    std::swap(branches_, branches_); // no-throw
    return *this;
}

```

Why it works: all throwing operations run on local copies first; the only mutations of `*this` are no-throw swaps. ⇒
Strong exception guarantee for free.

10. RAII (Resource Acquisition Is Initialisation)

- **Lock-guard example:** `std::lock_guard<std::mutex>(m)` unlocks `m` on *any* exit path — early return, exception, fall-through — because the destructor fires deterministically when the guard leaves scope.
- This is what makes C++ resource management *predictable*: you know exactly when and in what order resources are released. GC-based languages can't promise that.
- In HFT: every microsecond-jitter you eliminate from non-deterministic finalization is alpha. RAII is structural determinism.

11. Exceptions vs error codes

- Pre-2010s: thrown exceptions were slow even on the *non-throwing* path → many HFT shops banned them.
- Modern compilers (zero-cost EH model): exceptions cost **zero** on the non-throwing path; only the throwing path pays.
- **Guideline today:** use exceptions for *truly exceptional* conditions (OOM, file-not-found, programmer-bug). Use return values / `std::optional` / `std::expected` for *expected* failure modes (parse failures, partial reads).

12. Drawbacks (be honest)

- **Compile times** — header includes are textual substitution; modules (C++20) help.
- **Header/source split** — duplicated declarations, forward-decl gymnastics.
- **Slim stdlib** — no built-in networking (until C++23 `std::execution` + `std::print`), no GUI, no graphics. You will pull Boost/Asio/etc.
- **Build/dependency management** — no canonical package manager; CMake + `vcpkg`/Conan are the de-facto answer.

Concept → Repo file map (2nd-ed Packt code)

Section

File

What it shows

Section	File	What it shows
2.1 Zero-cost abstractions	code/abstractions.cpp	C vs C++ Hamlet counter — same machine code, different readability.
2.2 Memory layout	code/heap_allocations.cpp	Stack-allocated <code>Cars</code> and a contiguous <code>std::vector<Car></code> .
2.3 Value semantics	code/value_semantics.cpp	<code>Bagel(std::set<std::string>)</code> copies its toppings by value.
2.4 Const correctness	code/const_correctness.cpp	<code>Person::age()</code> <code>const</code> , <code>Team::leader()</code> <code>const</code> -overload pair.
2.5 References vs pointers	code/references.cpp	<code>get_volume(const Sphere&)</code> VS <code>get_volume(const Sphere*)</code> .
2.6 Strict interfaces	code/strict_interfaces.cpp (<i>stub — add yourself</i>)	<code>Boat/Engine += delete</code> copy.
2.8 Copy-and-swap	code/copy_and_swap.cpp (<i>stub — add yourself</i>)	<code>OakTree</code> with strong exception guarantee.
2.9 RAII	code/raii_lock_guard.cpp (<i>stub — add yourself</i>)	<code>std::lock_guard</code> deterministic unlock on every exit path.
— test runner	code/main.cpp	<code>RUN_ALL_TESTS()</code> — same as Packt.

Low-latency / HFT lens (extras beyond the book)

The book teaches general high-performance C++. As an HFT-oriented quant dev, layer these on top of Chapter 1:

- **Cache-line awareness** — `std::vector<Car>` $\approx$ great; `std::vector<std::unique_ptr<Car>>` $\approx$ pointer chase, $\approx$ Java. Prefer SoA over AoS for hot loops.
- **`alignas(64)`** — prevent false sharing on members touched by multiple threads (we already covered this with the `SpinBarrier` memory).
- **`noexcept` everywhere it's true** — enables small-buffer / inplace optimisations in containers; the `std::vector` reallocate-on-grow path picks move-construct only when the move is `noexcept`.
- **No exceptions in the hot path** — even with zero-cost EH, a thrown exception is non-deterministic; HFT shops typically use `std::expected` / status codes inside the hot path and exceptions only at startup/shutdown.
- **Owning vs non-owning is even more important at the metal** — every `shared_ptr` copy is two atomic ops (acquire + retain). On the hot path, a stale raw pointer + lifetime-by-construction wins.

Self-check questions (answer these without re-reading)

1. Write the C and C++ versions of `num_hamlet`. Which one produces fewer machine instructions?
2. Draw the memory layout of `std::vector<Car> cars(7)` in C++ and `ArrayList<Car> cars(7 elements)` in Java.

3. Why does `Bagel(std::set<std::string> ts)` end with `: toppings_(std::move(ts))` rather than `: toppings_(ts)`?
 4. Given `class Team { Person leader_; public: auto& leader() const; auto& leader(); };` — when is each overload selected?
 5. Convert this Java signature to the equivalent C++ signature that documents its nullability contract: `float getVolume(Sphere s).`
 6. What's the *strong* exception guarantee, and how does copy-and-swap achieve it?
 7. Why is `std::lock_guard` preferred over manual `m.lock()` / `m.unlock()` calls?
 8. Name one concrete HFT reason to avoid `std::shared_ptr` in the hot path.
-

Further reading

- Stroustrup, *A Tour of C++* 3rd ed — chapters 1–4 cover the same ground in 60 pages.
- Meyers, *Effective Modern C++* — items 11 (= `delete`), 25 (move-from rvalue refs), 41 (pass by value vs reference for sink params).
- CppCon 2014 — Chandler Carruth, "Efficiency with Algorithms, Performance with Data Structures" — visceral demonstration of the cache-locality point.